we recommend making the assertions as clear as possible, so we might re-write the test like this:

unittesting/test_roman4.rb

```
require_relative "roman3"
require "minitest/autorun"

class TestRoman < Minitest::Test
  def assert_roman_value(roman_numeral, arabic_numeral)
    assert_equal(roman_numeral, Roman.new(arabic_numeral).to_s)
  end

  def test_simple
    assert_roman_value("i", 1)
    assert_roman_value("ii", 2)
    assert_roman_value("iii", 3)
    assert_roman_value("iv", 4)
    assert_roman_value("ix", 9)
  end
end
```

produces:

```
Run options: --seed 51530
# Running:

.
Finished in 0.000429s, 2331.0023 runs/s, 11655.0115 assertions/s.
1 runs, 5 assertions, 0 failures, 0 errors, 0 skips
```

We think this does a good job separating the boilerplate action of the comparison from the data values we are trying to compare.

What else can we test? Well, the constructor of our `Roman` class checks that the number we pass in can be represented as a Roman number, throwing an exception if it can't. Let's test the exception:

unittesting/test_roman5.rb

```
require_relative "roman3"
require "minitest/autorun"

class TestRoman < Minitest::Test
  def assert_roman_value(roman_numeral, arabic_numeral)
    assert_equal(roman_numeral, Roman.new(arabic_numeral).to_s)
  end

  def test_simple
    assert_roman_value("i", 1)
    assert_roman_value("ii", 2)
    assert_roman_value("iii", 3)
    assert_roman_value("iv", 4)
    assert_roman_value("ix", 9)
  end

  def test_range
    # no exception for these two...
    Roman.new(1)
    Roman.new(4999)
    # but an exception for these
    assert_raises(RuntimeError) { Roman.new(0) }
```